

VELARIS — Manual de Usuario para Consultas

Velaris Dev Team

Versión 1.0 — 2026

Introducción

Este manual describe cómo ejecutar las consultas del sistema VELARIS en Supabase. Todas las queries se ejecutan desde el **SQL Editor** del proyecto en Supabase.

Cómo ejecutar una consulta

1. Ir a **SQL Editor** en el menú izquierdo
2. Click en **"New query"**
3. Pegar la query
4. Click en **"Run"** o presionar Ctrl + Enter
5. Los resultados aparecen en la parte inferior

QUERY 01 — Productos con Stock Bajo o Crítico

Muestra todos los productos activos con su nivel de stock y estado de alerta.

```
SELECT
  p.product_id,
  p.name,
  p.brand,
  p.model,
  c.name          AS category,
  p.current_stock,
  p.minimum_stock,
  CASE
    WHEN p.current_stock = 0                      THEN 'CRITICAL -
Out of stock'
    WHEN p.current_stock <= p.minimum_stock      THEN 'LOW - Below
minimum'
    WHEN p.current_stock <= p.minimum_stock * 2 THEN 'WARNING -
Near minimum'
    ELSE                                           'OK'
  END          AS stock_status,
  p.sale_price * p.current_stock AS stock_value
```

```
FROM velaris.products p
JOIN velaris.categories c ON p.category_id = c.category_id
WHERE p.active = TRUE
ORDER BY p.current_stock ASC;
```

Cuándo usarla: para identificar qué productos necesitan reabastecimiento urgente.

Columnas clave:

- stock_status — estado del stock (CRITICAL / LOW / WARNING / OK)
- stock_value — valor total del inventario disponible por producto

QUERY 02 — Productos Más Vendidos con Ingresos

Muestra los productos ordenados por ingresos totales generados.

```
SELECT
  p.product_id,
  p.name,
  p.brand,
  c.name AS category,
  COUNT(DISTINCT sd.sale_id) AS total_sales,
  SUM(sd.quantity) AS units_sold,
  SUM(sd.quantity * sd.unit_price) AS total_revenue,
  SUM(sd.quantity * (sd.unit_price - p.purchase_price)) AS
total_profit
FROM velaris.sales_details sd
JOIN velaris.products p ON sd.product_id = p.product_id
JOIN velaris.categories c ON p.category_id = c.category_id
GROUP BY p.product_id, p.name, p.brand, c.name
ORDER BY total_revenue DESC;
```

Cuándo usarla: para conocer qué productos generan más ingresos y ganancia.

Columnas clave:

- total_sales — número de ventas en que apareció el producto
- units_sold — unidades totales vendidas
- total_profit — ganancia total (precio venta - precio compra)

QUERY 03 — Empleados con Más Ventas

Muestra el desempeño de cada empleado en ventas.

```
SELECT
  e.employee_id,
  e.first_name || ' ' || e.last_name AS employee,
  e.position,
  COUNT(DISTINCT s.sale_id)          AS total_sales,
  SUM(sd.quantity)                   AS units_sold,
  SUM(sd.quantity * sd.unit_price)   AS total_revenue
FROM velaris.employees e
JOIN velaris.sales       s  ON e.employee_id = s.employee_id
JOIN velaris.sales_details sd ON s.sale_id     = sd.sale_id
GROUP BY e.employee_id, e.first_name, e.last_name, e.position
ORDER BY total_revenue DESC;
```

Cuándo usarla: para evaluar el rendimiento del equipo de ventas.

QUERY 04 — Órdenes de Compra Completas

Muestra todas las órdenes de compra con sus totales calculados y almacenados.

```
SELECT
  po.order_id,
  po.order_date,
  po.status,
  s.name          AS supplier,
  s.city,
  e.first_name || ' ' || e.last_name AS employee,
  COUNT(pod.detail_id)                AS total_lines,
  SUM(pod.quantity)                   AS total_units,
  SUM(pod.quantity * pod.unit_price)   AS calculated_total,
  po.total                            AS stored_total
FROM velaris.purchase_orders po
JOIN velaris.suppliers       s  ON po.supplier_id =
s.supplier_id
JOIN velaris.employees       e  ON po.employee_id =
e.employee_id
JOIN velaris.purchase_order_details pod ON po.order_id =
pod.order_id
GROUP BY po.order_id, po.order_date, po.status,
         s.name, s.city, e.first_name, e.last_name, po.total
```

```
ORDER BY po.order_date DESC;
```

Cuándo usarla: para revisar el historial de compras a proveedores.

Columnas clave:

- `calculated_total` — total calculado en tiempo real desde los detalles
- `stored_total` — total almacenado en la tabla (deben coincidir)

QUERY 05 — Clientes con Mayor Valor de Compra

Muestra los clientes ordenados por el total gastado.

```
SELECT
  cu.customer_id,
  cu.first_name || ' ' || cu.last_name AS customer,
  cu.email,
  COUNT(DISTINCT s.sale_id)           AS total_purchases,
  SUM(sd.quantity)                   AS total_units,
  SUM(sd.quantity * sd.unit_price)    AS total_spent,
  AVG(s.total)                       AS avg_purchase_value
FROM velaris.customers cu
JOIN velaris.sales      s ON cu.customer_id = s.customer_id
JOIN velaris.sales_details sd ON s.sale_id = sd.sale_id
GROUP BY cu.customer_id, cu.first_name, cu.last_name, cu.email
HAVING SUM(sd.quantity * sd.unit_price) > 0
ORDER BY total_spent DESC;
```

Cuándo usarla: para identificar los mejores clientes.

QUERY 06 — Resumen Mensual de Ventas con Crecimiento

Muestra las ventas por mes y el porcentaje de crecimiento respecto al mes anterior.

```
WITH monthly_sales AS (
  SELECT
    DATE_TRUNC('month', s.sale_date) AS month,
    COUNT(DISTINCT s.sale_id)        AS total_sales,
    SUM(sd.quantity)                 AS units_sold,
    SUM(sd.quantity * sd.unit_price) AS revenue
```

```

FROM velaris.sales s
JOIN velaris.sales_details sd ON s.sale_id = sd.sale_id
GROUP BY DATE_TRUNC('month', s.sale_date)
)
SELECT
  TO_CHAR(month, 'Month YYYY') AS period,
  total_sales,
  units_sold,
  revenue,
  LAG(revenue) OVER (ORDER BY month) AS previous_month_revenue,
  ROUND(
    (revenue - LAG(revenue) OVER (ORDER BY month)) /
    NULLIF(LAG(revenue) OVER (ORDER BY month), 0) * 100, 2
  ) AS growth_percentage
FROM monthly_sales
ORDER BY month;

```

Cuándo usarla: para analizar tendencias de ventas mes a mes.

Columnas clave:

- growth_percentage — crecimiento porcentual respecto al mes anterior (negativo = caída)

QUERY 07 — Productos Nunca Vendidos

Muestra productos activos que no tienen ninguna venta registrada.

```

SELECT
  p.product_id,
  p.name,
  p.brand,
  p.model,
  c.name AS category,
  p.current_stock,
  p.sale_price,
  p.created_at
FROM velaris.products p
JOIN velaris.categories c ON p.category_id = c.category_id
WHERE p.active = TRUE
  AND NOT EXISTS (
    SELECT 1
    FROM velaris.sales_details sd
  )

```

```
WHERE sd.product_id = p.product_id
)
ORDER BY p.created_at ASC;
```

Cuándo usarla: para identificar productos sin rotación que pueden representar inventario muerto.

QUERY 08 — Historial de Movimientos con Balance Corriente

Muestra todos los movimientos de inventario con el balance acumulado por producto.

```
SELECT
  m.movement_id,
  m.movement_date,
  p.name AS product,
  p.brand,
  m.movement_type,
  CASE
    WHEN m.movement_type IN ('entry', 'return', 'adjustment')
  THEN m.quantity
    WHEN m.movement_type =
'exit' THEN -m.quantity
  END AS stock_change,
  SUM(
    CASE
      WHEN m.movement_type IN ('entry', 'return', 'adjustment')
    THEN m.quantity
      WHEN m.movement_type =
'exit' THEN -m.quantity
    END
  ) OVER (
    PARTITION BY m.product_id
    ORDER BY m.movement_date
  ) AS running_balance,
  w.name AS warehouse,
  e.first_name || ' ' || e.last_name AS employee,
  m.sale_id,
  m.purchase_order_id,
  m.notes
FROM velaris.inventory_movements m
JOIN velaris.products p ON m.product_id = p.product_id
```

```
JOIN velaris.warehouses w ON m.warehouse_id = w.warehouse_id
JOIN velaris.employees e ON m.employee_id = e.employee_id
ORDER BY m.product_id, m.movement_date;
```

uándo usarla: para auditar el historial completo de movimientos de un producto y verificar su trazabilidad.

Columnas clave:

- stock_change — cantidad que sumó o restó ese movimiento
- running_balance — stock acumulado hasta ese momento
- sale_id — FK a la venta que originó el movimiento (si aplica)
- purchase_order_id — FK a la orden de compra que originó el movimiento (si aplica)

QUERY 09 — Análisis de Margen de Ganancia por Categoría

Muestra el margen promedio de ganancia por categoría de productos.

```
SELECT
  c.name      AS category,
  COUNT(p.product_id) AS total_products,
  ROUND(AVG(p.purchase_price), 2) AS avg_purchase_price,
  ROUND(AVG(p.sale_price), 2) AS avg_sale_price,
  ROUND(AVG(p.sale_price - p.purchase_price), 2) AS avg_margin,
  ROUND(
    AVG((p.sale_price - p.purchase_price) / p.purchase_price *
100), 2
  ) AS avg_margin_percentage,
  SUM(p.current_stock * p.sale_price) AS total_stock_value
FROM velaris.categories c
JOIN velaris.products p ON c.category_id = p.category_id
WHERE p.active = TRUE
GROUP BY c.name
ORDER BY avg_margin_percentage DESC;
```

Cuándo usarla: para identificar qué categorías son más rentables y cuál es el valor total del inventario por categoría.

QUERY 10 — Auditoría Completa de Cambios

Muestra el registro de auditoría de los últimos 90 días.

```
SELECT
  al.audit_id,
  al.recorded_at,
  al.affected_table,
  al.action,
  al.db_user,
  al.old_values,
  al.new_values,
  CASE
    WHEN al.action = 'INSERT' THEN al.new_values
    WHEN al.action = 'DELETE' THEN al.old_values
    WHEN al.action = 'UPDATE' THEN
      jsonb_build_object(
        'before', al.old_values,
        'after', al.new_values
      )
  END AS change_detail
FROM velaris.audit_log al
WHERE al.recorded_at >= NOW() - INTERVAL '90 days'
ORDER BY al.recorded_at DESC
LIMIT 50;
```

Cuándo usarla: para rastrear quién hizo qué cambio y cuándo. Útil para auditorías y resolución de inconsistencias.

Columnas clave:

- `change_detail`— muestra el antes y después del cambio en formato JSONB
- `db_user` — usuario de base de datos que ejecutó la acción

CRUD — Operaciones sobre Productos

Crear un producto

```
INSERT INTO velaris.products (name, model, brand, purchase_price,
  sale_price, minimum_stock, category_id)
VALUES ('Nombre', 'Modelo', 'Marca', 0000000, 0000000, 5, 1)
RETURNING product_id, name, brand, sale_price, current_stock;
```


Leer un producto por ID

```
SELECT p.*, c.name AS category
FROM velaris.products p
JOIN velaris.categories c ON p.category_id = c.category_id
WHERE p.product_id = 1;
```

Actualizar precio de un producto

```
UPDATE velaris.products
SET purchase_price = 0000000, sale_price = 0000000
WHERE product_id = 1
RETURNING product_id, name, purchase_price, sale_price;
```

Desactivar un producto (soft delete)

```
UPDATE velaris.products
SET active = FALSE
WHERE product_id = 1
RETURNING product_id, name, active;
```

No se recomienda eliminar productos físicamente ya que están referenciados en ventas y movimientos de inventario históricos.